

ReleaseProof

A change-aware, evidence-backed release gate that detects hidden regressions in a Cloud Run candidate before production traffic is shifted.

CATEGORY Future of Work and Enterprise Productivity	MVP SCOPE Cloud Run HTTP services; stable versus tagged candidate	DECISION PASS, BLOCK, or INCONCLUSIVE	AI ROLE Hypothesis formation and bounded experiment selection
---	---	---	---

1. Executive abstract

Modern deployment systems can run health checks, smoke tests, canaries, and user-configured verification jobs. The remaining problem is not the absence of tests; it is choosing the most informative test for a particular code or configuration change. A candidate may return HTTP 200 while introducing tail-latency under concurrency, breaking an API response contract, or failing only at input boundaries. Fixed verification often misses these change-specific risks until production traffic is exposed.

ReleaseProof adds a bounded investigation layer before traffic migration. Gemini reads a change summary and diff, states a falsifiable regression hypothesis, and selects up to two experiments from an allowlisted catalog. The same experiment is executed against the stable revision and a tagged candidate with zero user traffic. Deterministic code owns measurements, thresholds, and the final release verdict. Every input, plan, measurement, threshold, and outcome is retained as replayable evidence.

Core principle: AI plans the investigation; measured evidence makes the release decision.

2. Problem and target user

The primary user is a small platform or application team deploying HTTP services to Cloud Run without a dedicated site-reliability engineering function. Such teams can configure tests, but they may not anticipate which risk a specific change introduces. ReleaseProof targets the decision point immediately before traffic migration, when a stable and candidate revision both exist and a fast, bounded comparison is possible.

Observed gap	Consequence	ReleaseProof response
Health checks validate availability, not behavioral equivalence.	A healthy candidate can still be slower or incompatible.	Always compare the same probe on stable and candidate.
Fixed suites are broad but not change-aware.	Relevant tests may be omitted or buried under expensive checks.	Select experiments from diff-derived, falsifiable risks.
AI release advice can be persuasive but unverifiable.	Teams may trust fluent explanations instead of measurements.	Prevent Gemini from setting thresholds or overriding verdicts.
Canaries expose some users to the candidate.	A bad release can affect customers before analysis converges.	Probe a tagged, zero-traffic candidate before rollout.

3. Differentiation from existing deployment verification

ReleaseProof does not claim that Google Cloud lacks verification. Cloud Deploy already supports verification tasks, canary phases, and telemetry analysis. The proposed contribution is a change-specific planning and differential evidence layer that can eventually run as a Cloud Deploy verification task.

Capability	Conventional configured verification	ReleaseProof
Test choice	Defined in advance by the team	Selected per change from an allowlisted catalog
Comparison	Often candidate-only or telemetry-based	Paired stable-versus-candidate experiment
Timing	After deployment or during canary traffic	Before candidate receives production traffic
Decision ownership	Test exit code or configured analysis	Fixed policy over preserved raw evidence
AI contribution	Optional	Risk hypothesis and bounded experiment plan

4. Technical architecture

Firestore-ready evidence ledger
-> paired HTTP experiment runner -> deterministic evaluator -> Vertex AI Gemini planner
-> Cloud Run orchestrator -> Firebase-ready React UI

Component	Prototype responsibility	Production mapping
Web application	Select scenario, display change, plan, measurements, decision timeline, and evidence ID.	Firebase Hosting or Cloud Run
Orchestrator API	Validate bounded requests, call planner, execute experiments, calculate verdict, expose run records.	Cloud Run service
Gemini planner	Interpret diff, state risk, choose no more than two catalog experiments using structured output.	Gemini on Vertex AI
Experiment runner	Issue identical warmups and bounded probes to stable and candidate URLs.	Private Cloud Run service or orchestrator worker
Policy evaluator	Apply fixed contract, latency, error-rate, and completeness rules.	Deterministic application code
Evidence ledger	Persist request, plan, thresholds, measurements, verdict, timestamps, and run ID.	Firestore plus Cloud Storage and Logging

5. End-to-end analysis sequence

Step	Stage	What happens
1	Receive change	Service metadata, diff, stable URL, candidate URL, request path, expected contract, and execution budget.
2	Run fixed baseline	Health and smoke checks execute for every candidate; adaptive planning cannot remove them.
3	Form hypothesis	Gemini turns change evidence into a concrete statement that an experiment can support or refute.
4	Select experiment	Structured output is restricted to the catalog and revalidated server-side; maximum two adaptive tests.
5	Execute paired probes	Stable and candidate receive identical requests, warmups, trials, and request limits.
6	Evaluate deterministically	Fixed policies compare compatibility, errors, p95 latency, and evidence completeness.
7	Record decision	Any blocking failure yields BLOCK; insufficient evidence yields INCONCLUSIVE; otherwise PASS.

6. Bounded experiment catalog

Experiment	Risk addressed	MVP status
Health check	Revision unavailable or non-responsive	Implemented; mandatory baseline
API smoke	Primary endpoint fails for a representative request	Implemented; mandatory baseline
Contract compatibility	Removed fields, changed types, or incompatible response shape	Implemented and demonstrated
Bounded load	Tail-latency or error regression under concurrent requests	Implemented and demonstrated
Edge inputs	Empty, Unicode, zero, negative, or boundary input behavior	Cataloged; basic probe support
Payload size	Near-limit payload failure or disproportionate latency	Cataloged; basic probe support
Retry/idempotency	Duplicate side effects after repeated request	Cataloged; future domain-specific adapter
Dependency timeout	Slow upstream causes budget exhaustion or cascading failure	Cataloged; future fault adapter

7. Decision policy and trust boundary

PASS	Every executed blocking experiment passes fixed policy and sufficient repeated evidence exists.
BLOCK	At least one blocking experiment demonstrates a contract, error-rate, availability, or latency violation.
INCONCLUSIVE	Stable baseline is invalid, trials are incomplete, budget is exhausted, or the selected experiment lacks usable evidence.

Gemini may: interpret the change, choose allowlisted experiments, revise a hypothesis, and explain evidence. **Gemini may not:** invent measurements, issue arbitrary network requests, select its own threshold, execute shell commands, suppress failed evidence, or override the evaluator.

8. Demonstration scenarios

	Scenario A: concurrency regression	Scenario B: API contract regression
Change	Cloud Run concurrency increases from 4 to 32.	Response field total is renamed to amount and number becomes string.
Why smoke passes	Single requests remain fast and return HTTP 200.	Endpoint remains available and returns HTTP 200.
Gemini hypothesis	CPU/resource contention may increase candidate tail-latency or errors.	Existing clients may fail because a required field and type changed.

	Scenario A: concurrency regression	Scenario B: API contract regression
Selected test	Bounded paired load experiment.	Contract compatibility experiment.
Deterministic evidence	Stable p95 versus candidate p95 and error-rate delta.	Required fields and types compared with published contract and stable response.
Expected verdict	BLOCK when p95 regression exceeds 30% or error increase exceeds 2 percentage points.	BLOCK on a missing required field or incompatible type.

The second scenario is essential evidence that the planner is not blindly choosing the same test. A runtime configuration diff selects load; an application schema diff selects contract compatibility. In both cases the fixed baseline passes, so the adaptive experiment provides incremental value.

9. Prototype status and evaluation plan

IMPLEMENTED React UI, FastAPI API, planner, catalog, paired runner, evaluator, JSON ledger, and stable/candidate services	VERIFIED 8 backend tests, 8 demo-service tests, frontend build, and live HTTP validation of both scenarios	LOCAL MODE Deterministic planner supports development without cloud credentials	CLOUD MODE Vertex AI structured-output planner is implemented and activated by environment configuration
---	--	---	--

Evaluation measure	Question answered	Target for submission
Plan validity	Does every plan satisfy schema, allowlist, and budget?	100% of demo and held-out changes
Adaptive diversity	Do meaningfully different changes select different experiments?	At least two change classes and two plans
Baseline blind spot	Would health and smoke have passed without the adaptive experiment?	Yes for both principal scenarios
Repeatability	Do repeated paired trials produce the same decision?	Three repeated runs per scenario
Evidence integrity	Can every verdict be traced to raw measurement and fixed threshold?	Complete ledger for every run
Latency	Can the result fit a practical pre-traffic check?	Under 90 seconds within request budget

10. Expected judging alignment

Criterion	Weight	Evidence in prototype
Technical merit and GenAI	40%	Schema-constrained Gemini planning, real paired execution, deterministic policy, and evidence ledger.
Problem alignment and impact	25%	Prevents hidden regressions before user traffic and supports teams without specialist release engineering.
Innovation and creativity	25%	Change-specific, budgeted experiment selection plus pre-traffic stable-versus-candidate comparison.
User experience and design	10%	Two-click workflow, visible hypothesis, measurements, verdict, and decision timeline.

11. Limitations and roadmap

CURRENT LIMITATIONS	NEXT DEVELOPMENT STEPS
- Narrow HTTP-only experiment catalog. - Demo thresholds are generic, not organization-calibrated. - Local ledger is JSON rather than immutable cloud storage. - Statistical treatment is intentionally simple. - Prototype does not automatically control production rollout.	1. Deploy stable/candidate revisions and orchestrator to Cloud Run. 2. Execute the Vertex AI Gemini planner with real credentials. 3. Persist evidence in Firestore and export metrics to Cloud Monitoring. 4. Add authenticated tagged-revision discovery and project allowlists. 5. Package ReleaseProof as a Cloud Deploy verification task.

12. Conclusion and references

ReleaseProof is deliberately not an autonomous deployment agent. It is a constrained release investigator that converts an AI-selected hypothesis into paired, reproducible evidence and lets deterministic policy own the outcome. The prototype's success criterion is concrete: demonstrate a candidate that normal fixed verification would accept, but a change-aware experiment blocks before users are exposed.

Primary references: [AI Builder Cup 2026 judging criteria](#); [Cloud Deploy verification](#); [Cloud Deploy analysis](#); [Cloud Run revision traffic and tags](#).